

```

"""
PathForge V2 - Solutions by Family
=====

One reference implementation per family,
matching the "What the code is
doing" description in the human-review
sheet. Multiple families per
problem = multiple genuinely different
algorithmic approaches.

Decision key: APPROVED / CORRECTED LABEL /
REJECTED
(See PathForge_V2_Review_Decisions.docx for
the full remark on each.)
"""

from collections import deque, defaultdict,
Counter
from typing import List, Optional

#
=====
=====
# LC 1 -- Two Sum
#
=====
=====

def two_sum_fam1_hash_lookup(nums:

```

```

List[int], target: int) -> List[int]:
    """lc1_fam1 -- APPROVED (hash_lookup)
    Dictionary stores seen values, looks up
    the needed complement.
    """
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
    return []

def two_sum_fam2_brute_force_for(nums:
List[int], target: int) -> List[int]:
    """lc1_fam2 -- CORRECTED LABEL:
    brute_force
    Nested for-loops check every pair
    directly.
    """
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

def two_sum_fam3_brute_force_while(nums:
List[int], target: int) -> List[int]:
    """lc1_fam3 -- CORRECTED LABEL:
    brute_force
    Nested while-loops; same brute-force

```

```

idea, different syntax.
"""
n = len(nums)
i = 0
while i < n:
    j = i + 1
    while j < n:
        if nums[i] + nums[j] == target:
            return [i, j]
        j += 1
    i += 1
return []

```

```

def two_sum_fam4_sorted_two_pointer(nums:
List[int], target: int) -> List[int]:
    """lc1_fam4 -- CORRECTED LABEL:
two_pointers_opposite
    Sorts (index, value) pairs, then
low/high pointers move inward.
    """
    indexed = sorted(range(len(nums)),
key=lambda i: nums[i])
    lo, hi = 0, len(indexed) - 1
    while lo < hi:
        s = nums[indexed[lo]] +
nums[indexed[hi]]
        if s == target:
            return sorted([indexed[lo],
indexed[hi]])
        elif s < target:
            lo += 1
        else:
            hi -= 1

```

```

        return []

#
=====
=====
# LC 21 -- Merge Two Sorted Lists
#
=====
=====

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_lists_fam1_forward_pointer(l1,
l2):
    """lc21_fam1 -- APPROVED
(forward_pointer_advance)
    Iteratively chooses the smaller current
node and advances pointers.
    """
    if l1 is None:
        return l2
    if l2 is None:
        return l1
    head = l1 if l1.val <= l2.val else l2
    if head is l1:
        l1 = l1.next
    else:
        l2 = l2.next
    curr = head

```

```

while l1 and l2:
    if l1.val <= l2.val:
        curr.next = l1
        l1 = l1.next
    else:
        curr.next = l2
        l2 = l2.next
    curr = curr.next
curr.next = l1 if l1 else l2
return head

```

```

def merge_lists_fam2_sort_and_rebuild(l1,
l2):

```

```

    """lc21_fam2 -- CORRECTED LABEL:
sort_and_rebuild
    Collects all node values, sorts them,
rebuilds a new linked list.
    """

```

```

    values = []
    for node in (l1, l2):
        while node:
            values.append(node.val)
            node = node.next
    values.sort()
    dummy = ListNode()
    curr = dummy
    for v in values:
        curr.next = ListNode(v)
        curr = curr.next
    return dummy.next

```

```

def merge_lists_fam3_dummy_node_merge(l1,

```

```

l2):
    """lc21_fam3 -- APPROVED
(forward_pointer_advance)
    Iterative dummy-node merge: compare
heads, attach smaller, advance.
    """
    dummy = ListNode()
    curr = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            curr.next = l1
            l1 = l1.next
        else:
            curr.next = l2
            l2 = l2.next
        curr = curr.next
    curr.next = l1 if l1 else l2
    return dummy.next

def merge_lists_fam4_recursive_merge(l1,
l2):
    """lc21_fam4 -- CORRECTED LABEL:
recursive_merge
    Recursively compares the two current
nodes and links the smaller one.
    """
    if l1 is None:
        return l2
    if l2 is None:
        return l1
    if l1.val <= l2.val:
        l1.next =
merge_lists_fam4_recursive_merge(l1.next,

```

```

12)
    return l1
else:
    l2.next =
merge_lists_fam4_recursive_merge(l1,
l2.next)
    return l2

#
=====
=====
# LC 46 -- Permutations
#
=====
=====

def permute_fam1_iterative_insertion(nums:
List[int]) -> List[List[int]]:
    """lc46_fam1 -- CORRECTED LABEL:
iterative_insertion
    Builds permutations iteratively by
inserting each new element
    into every position of every existing
permutation.
    """
    result = [[]]
    for n in nums:
        new_result = []
        for perm in result:
            for i in range(len(perm) + 1):
                new_result.append(perm[:i]
+ [n] + perm[i:])
        result = new_result

```

```

        return result

def permute_fam2_backtracking_path(nums:
List[int]) -> List[List[int]]:
    """lc46_fam2 -- APPROVED
(dfs_backtracking)
    Recursive backtracking builds a path
and skips elements already used.
    """
    result = []

    def backtrack(path):
        if len(path) == len(nums):
            result.append(path[:])
            return
        for n in nums:
            if n in path:
                continue
            path.append(n)
            backtrack(path)
            path.pop()

    backtrack([])
    return result

def
permute_fam3_backtracking_used_array(nums:
List[int]) -> List[List[int]]:
    """lc46_fam3 -- APPROVED
(dfs_backtracking)
    Recursive backtracking with an explicit
used[] array and state

```

```

restoration.
"""
result = []
used = [False] * len(nums)
path = []

def backtrack():
    if len(path) == len(nums):
        result.append(path[:])
        return
    for i, n in enumerate(nums):
        if used[i]:
            continue
        used[i] = True
        path.append(n)
        backtrack()
        path.pop()
        used[i] = False

backtrack()
return result

```

```

#
=====
=====
# LC 70 -- Climbing Stairs
#
=====
=====

def climb_stairs_fam1_dp_bottom_up(n: int)
-> int:
    """lc70_fam1 -- APPROVED (dp_bottom_up)

```

```

    Bottom-up DP array: dp[i] = dp[i-1] +
dp[i-2].
    """
    if n <= 2:
        return n
    dp = [0] * (n + 1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

def climb_stairs_fam2_dp_top_down(n: int) -
> int:
    """lc70_fam2 -- CORRECTED LABEL:
dp_top_down
    Top-down recursion with memoization in
a dictionary.
    """
    memo = {}

    def helper(k):
        if k <= 2:
            return k
        if k in memo:
            return memo[k]
        memo[k] = helper(k - 1) + helper(k
- 2)
        return memo[k]

    return helper(n)

def climb_stairs_fam3_rolling_dp(n: int) ->

```

```

int:
    """lc70_fam3 -- CORRECTED LABEL:
dp_bottom_up
    Iterative two-variable rolling DP
without a full DP array
    (space-optimized bottom-up).
    """
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1

#
=====
=====
# LC 102 -- Binary Tree Level Order
Traversal
#
=====
=====

class TreeNode:
    def __init__(self, val=0, left=None,
right=None):
        self.val = val
        self.left = left
        self.right = right

def level_order_fam1_bfs_queue(root) ->
List[List[int]]:

```

```

"""lc102_fam1 -- APPROVED
(bfs_shortest_path)
    Queue-based level-order traversal
    (BFS), one level at a time.
"""
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result

def level_order_fam2_recursive_dfs(root) ->
List[List[int]]:
    """lc102_fam2 -- CORRECTED LABEL:
    recursive_dfs_by_depth
    Recursive DFS records node values by
    depth.
    """
    result = []

    def dfs(node, depth):
        if not node:
            return

```

```

        if depth == len(result):
            result.append([])
            result[depth].append(node.val)
            dfs(node.left, depth + 1)
            dfs(node.right, depth + 1)

    dfs(root, 0)
    return result

#
=====
=====
# LC 125 -- Valid Palindrome
#
=====
=====

def is_palindrome_fam1_two_pointers(s: str)
-> bool:
    """lc125_fam1 -- APPROVED
    (two_pointers_opposite)
    Two indices at both ends, skip invalid
    chars, compare, move inward.
    """
    lo, hi = 0, len(s) - 1
    while lo < hi:
        while lo < hi and not
s[lo].isalnum():
            lo += 1
        while lo < hi and not
s[hi].isalnum():
            hi -= 1
        if s[lo].lower() != s[hi].lower():

```

```

        return False
    lo += 1
    hi -= 1
    return True

def is_palindrome_fam2_clean_and_reverse(s:
str) -> bool:
    """lc125_fam2 -- CORRECTED LABEL:
clean_and_compare_reverse
    Cleans the string into a list, then
compares it with its reverse.
    """
    cleaned = [c.lower() for c in s if
c.isalnum()]
    return cleaned == cleaned[::-1]

def
is_palindrome_fam3_recursive_two_pointers(s
: str) -> bool:
    """lc125_fam3 -- CORRECTED LABEL:
two_pointers_opposite (recursive)
    Cleans the string, then recursively
compares left/right moving inward.
    """
    cleaned = [c.lower() for c in s if
c.isalnum()]

    def helper(lo, hi):
        if lo >= hi:
            return True
        if cleaned[lo] != cleaned[hi]:
            return False

```

```

        return helper(lo + 1, hi - 1)

    return helper(0, len(cleaned) - 1)

#
=====
=====
# LC 209 -- Minimum Size Subarray Sum
#
=====
=====

def
min_subarray_len_fam1_sliding_window(target
: int, nums: List[int]) -> int:
    """lc209_fam1 -- APPROVED
    (sequential_accumulation + sliding_window)
    Variable sliding window: expand right,
    shrink from left while
    the sum is large enough.
    """
    left = 0
    total = 0
    best = float('inf')
    for right, val in enumerate(nums):
        total += val
        while total >= target:
            best = min(best, right - left +
1)
            total -= nums[left]
            left += 1
    return 0 if best == float('inf') else
best

```

```

def
min_subarray_len_fam2_brute_force(target:
int, nums: List[int]) -> int:
    """lc209_fam2 -- CORRECTED LABEL:
brute_force
    Nested loops accumulate each subarray
sum, track shortest valid one.
    """
    n = len(nums)
    best = float('inf')
    for i in range(n):
        total = 0
        for j in range(i, n):
            total += nums[j]
            if total >= target:
                best = min(best, j - i + 1)
                break
    return 0 if best == float('inf') else
best

```

```

def
min_subarray_len_fam3_prefix_sum(target:
int, nums: List[int]) -> int:
    """lc209_fam3 -- CORRECTED LABEL:
sequential_accumulation
    Builds a prefix-sum array, then checks
prefix differences with
    nested loops.
    """
    n = len(nums)
    prefix = [0] * (n + 1)

```

```

    for i in range(n):
        prefix[i + 1] = prefix[i] + nums[i]
    best = float('inf')
    for i in range(n):
        for j in range(i, n):
            if prefix[j + 1] - prefix[i] >=
target:
                best = min(best, j - i + 1)
                break
    return 0 if best == float('inf') else
best

#
=====
=====
# LC 242 -- Valid Anagram
#
=====
=====

def is_anagram_fam1_count_array(s: str, t:
str) -> bool:
    """lc242_fam1 -- APPROVED
(frequency_counting)
    Fixed-size count array,
increments/decrements character
frequencies.
    """
    if len(s) != len(t):
        return False
    counts = [0] * 26
    for cs, ct in zip(s, t):
        counts[ord(cs) - ord('a')] += 1

```

```

        counts[ord(ct) - ord('a')] -= 1
    return all(c == 0 for c in counts)

def is_anagram_fam2_counter_equality(s:
str, t: str) -> bool:
    """lc242_fam2 -- REJECTED
    Family coherence could not be
    confirmed: sheet notes zero-evidence
    members alongside one Counter-equality
    member, so it is not
    verifiable that all members share one
    algorithmic approach.
    Shown form (Counter equality) for
    reference only.
    """
    return Counter(s) == Counter(t)

def is_anagram_fam3_manual_dict(s: str, t:
str) -> bool:
    """lc242_fam3 -- APPROVED
    (frequency_counting)
    Manual dictionary counts characters,
    decrements while checking
    the second string.
    """
    if len(s) != len(t):
        return False
    counts = {}
    for c in s:
        counts[c] = counts.get(c, 0) + 1
    for c in t:
        if c not in counts or counts[c] ==

```

```

0:
    return False
    counts[c] -= 1
    return True

#
=====
=====
# LC 547 -- Number of Provinces
#
=====
=====

def find_circle_num_fam1_bfs(is_connected:
List[List[int]]) -> int:
    """lc547_fam1 -- CORRECTED LABEL:
bfs_shortest_path
    BFS over the adjacency matrix using a
queue and visited array.
    """
    n = len(is_connected)
    visited = [False] * n
    provinces = 0
    for i in range(n):
        if visited[i]:
            continue
        provinces += 1
        queue = deque([i])
        visited[i] = True
        while queue:
            node = queue.popleft()
            for j in range(n):
                if is_connected[node][j] ==

```

```

1 and not visited[j]:
    visited[j] = True
    queue.append(j)
    return provinces

def find_circle_num_fam2_dfs(is_connected:
List[List[int]]) -> int:
    """lc547_fam2 -- CORRECTED LABEL:
recursive_dfs_traversal
    Recursive DFS over the adjacency matrix
using a visited array.
    """
    n = len(is_connected)
    visited = [False] * n

    def dfs(node):
        visited[node] = True
        for j in range(n):
            if is_connected[node][j] == 1
and not visited[j]:
                dfs(j)

    provinces = 0
    for i in range(n):
        if not visited[i]:
            provinces += 1
            dfs(i)
    return provinces

def
find_circle_num_fam3_union_find(is_connecte
d: List[List[int]]) -> int:

```

```

"""lc547_fam3 -- APPROVED (union_find)
Union-Find/disjoint-set with parent
array and find/union operations.
"""
n = len(is_connected)
parent = list(range(n))

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[ra] = rb

for i in range(n):
    for j in range(n):
        if is_connected[i][j] == 1:
            union(i, j)

return len({find(i) for i in range(n)})

#
=====
=====
# LC 560 -- Subarray Sum Equals K
#
=====
=====

```

```

def subarray_sum_fam1_brute_force(nums:
List[int], k: int) -> int:
    """lc560_fam1 -- CORRECTED LABEL:
brute_force
    Nested loops accumulate each subarray
total, count those equal to k.
    """
    n = len(nums)
    count = 0
    for i in range(n):
        total = 0
        for j in range(i, n):
            total += nums[j]
            if total == k:
                count += 1
    return count

def subarray_sum_fam2_prefix_hashmap(nums:
List[int], k: int) -> int:
    """lc560_fam2 -- APPROVED
(frequency_counting +
sequential_accumulation)
    Running prefix sum plus hash-map
frequency of previous prefix sums.
    """
    count = 0
    total = 0
    freq = defaultdict(int)
    freq[0] = 1
    for n in nums:
        total += n
        count += freq[total - k]
        freq[total] += 1

```

```

        return count

def
subarray_sum_fam3_prefix_dict_getset(nums:
List[int], k: int) -> int:
    """lc560_fam3 -- CORRECTED LABEL:
frequency_counting +
sequential_accumulation
    Same core idea as fam2: running prefix
sum plus dictionary
    get/set for prefix frequencies.
    """
    count = 0
    total = 0
    freq = {0: 1}
    for n in nums:
        total += n
        count += freq.get(total - k, 0)
        freq[total] = freq.get(total, 0) +
1
    return count

def
subarray_sum_fam4_prefix_array_nested(nums:
List[int], k: int) -> int:
    """lc560_fam4 -- CORRECTED LABEL:
sequential_accumulation
    Explicit prefix-sum array, then nested
loops compare prefix
differences.
    """
    n = len(nums)

```

```

    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i + 1] = prefix[i] + nums[i]
    count = 0
    for i in range(n):
        for j in range(i, n):
            if prefix[j + 1] - prefix[i] ==
k:
                count += 1
    return count

#
=====
=====
# LC 704 -- Binary Search
#
=====
=====

def
search_fam1_iterative_binary_search(nums:
List[int], target: int) -> int:
    """lc704_fam1 -- APPROVED
(binary_search)
    Iterative binary search with lo/hi/mid
and interval narrowing.
    """
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:

```

```
        lo = mid + 1
    else:
        hi = mid - 1
    return -1
```

```
def search_fam2_bisect_left(nums:
List[int], target: int) -> int:
    """lc704_fam2 -- CORRECTED LABEL:
    binary_search (library-based)
    Calls the standard-library bisect_left
    operation, which performs
    binary search internally, to locate the
    insertion/search position.
    """
    import bisect
    i = bisect.bisect_left(nums, target)
    if i < len(nums) and nums[i] == target:
        return i
    return -1
```

```
def
search_fam3_recursive_binary_search(nums:
List[int], target: int) -> int:
    """lc704_fam3 -- CORRECTED LABEL:
    binary_search (recursive)
    Recursive binary search that halves the
    search interval each call.
    """
    def helper(lo, hi):
        if lo > hi:
            return -1
        mid = (lo + hi) // 2
```

```

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            return helper(mid + 1, hi)
        else:
            return helper(lo, mid - 1)

    return helper(0, len(nums) - 1)

#
=====
# LC 3236 -- Smallest Missing Integer
Greater Than Sequential Prefix Sum
#
=====

def missing_integer_fam1_running_sum(nums:
List[int]) -> int:
    """lc3236_fam1 -- APPROVED
    (sequential_accumulation)
    Computes a running sum over the leading
    consecutive sequence,
    then increments the candidate while it
    is present.
    """
    total = nums[0]
    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            total += nums[i]
        else:
            break

```

```

        candidate = total + 1
        num_set = set(nums)
        while candidate in num_set:
            candidate += 1
        return candidate

def
missing_integer_fam2_prefix_array_set(nums:
List[int]) -> int:
    """lc3236_fam2 -- APPROVED
(sequential_accumulation)
    Builds a prefix array over the leading
consecutive sequence,
    then checks candidate membership with a
set.
    """
    prefix = [nums[0]]
    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            prefix.append(prefix[-1] +
nums[i])
        else:
            break
    candidate = prefix[-1] + 1
    num_set = set(nums)
    while candidate in num_set:
        candidate += 1
    return candidate

```